

TYPESCRIPT E REACT

TYPES PARA REACT

Crie o projeto, compreenda componentes e rotas e modele props e dados com contratos claros.



Vite



TypeScript



React



React Router

Alexandre Carlos de Jesus

github.com/alecarlosjesus

linkedin.com/in/alexandre-carlos-de-jesus

Versões e procedimentos validados em agosto de 2026

© 2026 Alexandre Carlos de Jesus - Todos os direitos reservados.

ANTES DE COMEÇAR

Como usar esta apostila

ROTA DE ESTUDO Leia uma explicação por vez, digite o exemplo e compare o resultado com o quadro Confira o resultado. Quando houver erro proposital, leia a mensagem antes de corrigir. O objetivo não é decorar símbolos: é entender o que cada parte comunica.

ENTENDA OS TERMOS ANTES DE USÁ-LOS

Palavras que você encontrará

Termo	Explicação em linguagem simples
Vite	Ferramenta instalada no projeto para iniciar o servidor local e preparar o build.
create-vite	Gerador que cria a pasta e os arquivos iniciais do projeto.
Template	Modelo escolhido pelo gerador. react-ts significa React com TypeScript.
Dependência	Pacote externo registrado no projeto e baixado pelo npm.
Componente	Parte da interface descrita por uma função com nome iniciado por letra maiúscula.
JSX	JavaScript XML: sintaxe semelhante a HTML usada para descrever a interface.
Props	Objeto de entrada usado para o componente pai enviar dados ao componente filho.
Rota	Regra que relaciona um endereço do navegador a um componente de página.
Contrato	Descrição dos nomes e tipos que um objeto deve possuir.

O QUE SERÁ FEITO

Organização da aula

Item	Definição
Duração estimada	300 a 360 minutos, organizados em dois blocos de estudo
Pré-requisitos	B01 concluída; Node.js 24 LTS e npm funcionando; conhecimento básico de funções e objetos.
Resultado esperado desta aula	Projeto Vite + React + TypeScript com componentes tipados, páginas navegáveis e catálogo organizado por responsabilidades.
Versões e base usadas	create-vite 9.1, Vite 8.2, TypeScript 7.0, React 19.2 e React Router 8.3; versões conferidas em agosto de 2026.
Arquivos trabalhados	package.json, src/App.tsx, src/main.tsx, src/pages/*.tsx, src/types/produto.ts e src/components/*.tsx

AO FINAL DESTA AULA

O que você aprenderá

Você conseguirá criar o projeto React com TypeScript e explicar de onde cada arquivo surgiu.

Você conseguirá distinguir o gerador create-vite da ferramenta Vite instalada no projeto.

Você conseguirá explicar componentes como unidades de responsabilidade, composição e reutilização.

Você conseguirá criar, exportar, importar e combinar componentes funcionais antes de adicionar props.

Você conseguirá explicar o fluxo de props entre componente pai e componente filho.

Você conseguirá modelar props e dados com type, interface, unions, literais e propriedades opcionais.

Você conseguirá usar callbacks, children, listas, intersections, verificação de alternativas e generics.

Você conseguirá criar rotas estáticas e dinâmicas com Link, useParams e useNavigate.

VEJA O ASSUNTO COMO UM TODO

Como as partes se conectam



A URL seleciona uma página; a página combina componentes; props levam dados tipados até cada parte da interface.

COMECE PELA ORIGEM DOS ARQUIVOS

1. Criar o projeto e entender as versões

Nesta apostila, você não receberá uma pasta pronta. O projeto será criado pelo terminal para que App.tsx, main.tsx e package.json não pareçam arquivos surgidos do nada.

Boilerplate e template não são a aplicação pronta

Boilerplate é uma base inicial com configurações que seriam repetidas em muitos projetos. Template significa modelo. O create-vite usa o template react-ts para gerar uma base com React e TypeScript. Essa base já funciona, mas ainda contém apenas uma demonstração; o catálogo será construído por você.

TRADUÇÃO PRÁTICA create-vite é o gerador; react-ts é o modelo; a pasta produzida é a base inicial; TechStore é a aplicação que construiremos sobre essa base.

Por que aparece o número 9 se o Vite é 8?

Existem dois pacotes com funções diferentes. O comando de criação usa create-vite, enquanto o projeto usa vite. Como os pacotes são independentes, seus números de versão não precisam coincidir.

Pacote conferido	Versão em 11/08/2026	Papel
create-vite	9.1.2	Cria os arquivos iniciais e termina seu trabalho.
vite	8.2.1	Permanece no projeto e executa dev e build.
react	19.2.8	Biblioteca usada para construir a interface.
react-router	8.3.0	Biblioteca atual de rotas para aplicações React.
typescript	7.0.2	Verifica os tipos durante o desenvolvimento.

ATENÇÃO O número mostrado depois de npm create vite@latest pode ser 9.1.x porque ele pertence ao create-vite. Para descobrir a versão do Vite instalada no projeto, use npx vite --version dentro da pasta criada.

Etapa 1 - verificar Node.js e npm

Abra o terminal do Visual Studio Code. npm é o gerenciador de pacotes instalado junto com o Node.js. Antes de criar o projeto, confirme que os dois comandos respondem.

TERMINAL

```
node --version
npm --version
```

CONFIRA O RESULTADO Node.js mostra v24.x e npm mostra uma versão válida. Se algum comando não for reconhecido, revise a A01 antes de continuar.

Etapa 2 - gerar a base React com TypeScript

TERMINAL - CRIAÇÃO DIRETA

```
npm create vite@latest techstore-types -- --template react-ts
```

Trecho	Leitura
npm create	Pede ao npm que execute um gerador de projeto.
vite@latest	Seleciona a versão estável mais recente do gerador create-vite.
techstore-types	Define o nome da pasta que será criada.
--	Entrega ao gerador as opções escritas depois deste separador.
--template react-ts	Escolhe React com TypeScript sem abrir perguntas no terminal.

A forma interativa é `npm create vite@latest`. Interativa significa que o terminal pergunta o nome, a tecnologia e a variante. Nesse caso, escolha React e depois TypeScript. Execute apenas uma forma de criação.

Etapa 3 - instalar e confirmar

TERMINAL

```
cd techstore-types
npm install
npm install --save-dev typescript@latest
npm install react-router
```

`cd` entra na pasta. O primeiro `npm install` lê `package.json` e baixa as dependências para `node_modules`. O segundo atualiza a ferramenta TypeScript usada nesta coleção. O último adiciona a versão atual do React Router.

TERMINAL - CONFERIR O PROJETO REAL

```
npx vite --version
npx tsc --version
npm list react react-router --depth=0
```

CONFIRA O RESULTADO Os comandos mostram Vite 8.2.x, TypeScript 7.0.x, React 19.2.x e React Router 8.3.x. O último número pode aumentar com correções futuras.

Etapa 4 - iniciar o servidor local

TERMINAL

```
npm run dev
```

O script `dev` inicia o servidor de desenvolvimento. Servidor local é um programa executado no seu computador para mostrar a aplicação enquanto o código está sendo editado. Abra o endereço apresentado, normalmente `http://localhost:5173`.

ATENÇÃO O terminal permanece ocupado enquanto o servidor está ativo. Pressione `Ctrl + C` para encerrá-lo. Fechar a aba do navegador não encerra o servidor.

SAIBA QUEM CHAMA QUEM

2. Ler a estrutura antes de limpar

ESTRUTURA INICIAL DO TEMPLATE

```
techstore-types/
├── index.html
├── package.json
├── public/
├── src/
│   ├── assets/
│   ├── App.css
│   ├── App.tsx
│   ├── index.css
│   └── main.tsx
├── tsconfig.json
└── vite.config.ts
```

Arquivo	Responsabilidade
package.json	Registra dependências e scripts como dev e build.
index.html	Contém o elemento root, ponto em que a interface React será inserida.
src/main.tsx	Inicia o React, aplica configurações gerais e renderiza App.
src/App.tsx	Componente principal que organiza as partes visíveis da aplicação.
src/index.css	Guarda estilos gerais usados por toda a aplicação.
vite.config.ts	Configura o Vite e o suporte do React.

O caminho inicial pode ser lido assim: index.html oferece o ponto root; main.tsx encontra esse ponto; main.tsx pede ao React que renderize App; App combina a interface exibida.

Criar uma base limpa sem quebrar o projeto

Uma base limpa é uma aplicação funcionando sem a demonstração do Vite. Ela não é uma pasta vazia. Substitua App.tsx e index.css pelos exemplos abaixo.

SRC/APP.TSX

```
function App() {
  return (
    <main>
      <h1>TechStore 2026</h1>
      <p>Projeto React funcionando.</p>
    </main>
  );
}

export default App;
```

SRC/INDEX.CSS

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
}

button,
input {
  font: inherit;
}
```

App.css e os arquivos de demonstração em src/assets podem ser removidos depois que App.tsx deixar de importá-los. main.tsx, package.json, index.html, tsconfig.json e vite.config.ts continuam necessários.

CONFIRA O RESULTADO A página mostra TechStore 2026 e Projeto React funcionando. O terminal não apresenta erro.

VAMOS PRATICAR Sem olhar a tabela, explique em voz alta a sequência index.html → main.tsx → App.tsx. Se algum arquivo ainda parecer sem função, releia esta seção antes de criar componentes.

ENTENDA A IDEIA ANTES DA SINTAXE

3. Componente: a unidade central do React

Uma interface inteira em um único App.tsx funciona no começo, mas logo mistura menu, produtos, mensagens, botões e regras diferentes no mesmo lugar. Componentizar significa separar a interface em partes com responsabilidades compreensíveis.

Responsabilidade é o trabalho principal de uma parte. Um Cabeçalho apresenta identidade e navegação. Uma ListaProdutos organiza vários produtos. Um CardProduto apresenta um produto. Quando conseguimos dar um nome claro à responsabilidade, encontramos um possível componente.

Parte da TechStore	Responsabilidade	Possível componente
Topo	Apresentar o nome e a navegação.	Cabecalho
Conteúdo inicial	Explicar o objetivo da página.	Apresentacao
Conjunto de produtos	Percorrer e organizar os produtos.	ListaProdutos
Produto individual	Mostrar nome, preço, status e ações.	CardProduto
Página completa	Combinar as partes relacionadas à rota inicial.	Inicio

Três perguntas para decidir uma separação

1. Consigo dar a essa parte um nome que revele seu trabalho?
2. Essa parte possui conteúdo ou comportamento que pode mudar sem exigir a leitura da página inteira?
3. Separá-la deixará o componente pai mais fácil de entender ou permitirá reutilização?

ATENÇÃO Nem toda div precisa virar componente. A separação deve melhorar a leitura, isolar uma responsabilidade ou permitir reutilização. Um arquivo com dezenas de componentes minúsculos também pode dificultar o entendimento.

Anatomia do primeiro componente

SRC/COMPONENTS/TITULOCATALOGO.TSX

```
export function TituloCatalogo() {
  return <h2>Produtos em destaque</h2>;
}
```

Parte	Significado
export	Permite que outro arquivo importe este componente.
function	Declara uma função JavaScript.
TituloCatalogo	Nome em PascalCase: cada palavra começa com letra maiúscula.
()	Local onde serão recebidas as props quando elas existirem.
return	Entrega ao React a interface que deve ser exibida.
...	JSX que descreve o elemento mostrado no navegador.

Um componente React é uma função usada pelo React para descrever uma parte da interface. O nome começa com letra maiúscula para diferenciá-lo de uma tag HTML. `<section>` representa um elemento do navegador; `<TituloCatalogo />` representa um componente criado no projeto.

Escrita	Quem reconhece	Exemplo
minúscula	O navegador reconhece como elemento HTML.	<code><header></code> , <code><main></code> , <code><button></code>
Maiúscula	O React procura uma função com esse nome.	<code><Cabecalho /></code> , <code><CardProduto /></code>

Compor é montar uma parte com outras partes

SRC/APP.TSX

```
import { TituloCatalogo } from "../components/TituloCatalogo";

function App() {
  return (
    <main>
      <h1>TechStore 2026</h1>
      <TituloCatalogo />
    </main>
  );
}

export default App;
```

`import` localiza o código exportado. `<TituloCatalogo />` solicita sua renderização dentro de `App`. `App` é pai porque contém `TituloCatalogo`; `TituloCatalogo` é filho porque foi colocado dentro de `App`. Essa relação descreve a posição na árvore, não importância.

ÁRVORE SIMPLIFICADA DA INTERFACE

```
App
├── main
│   ├── h1
│   └── TituloCatalogo
│       └── h2
```

VOCÊ SABIA? Um componente não é apenas um arquivo. O componente é a função. Para facilitar a organização, normalmente colocamos o principal componente exportado em um arquivo com o mesmo nome.

► VÍDEO DE APOIO [React 19 Tutorial - Components, Codevolution \(7 min 18 s\)](#)

Revisão visual qualificada sobre componentes. O vídeo está em inglês; ative as legendas automáticas em português. Assista depois de ler esta seção.

VAMOS PRATICAR Crie `src/components/MensagemBoasVindas.tsx`, faça o componente retornar um parágrafo e use `<MensagemBoasVindas />` dentro de `App`. Depois desenhe a nova árvore em três linhas.

DADOS DESCEM DO PAI PARA O FILHO

4. Props: transformar uma peça fixa em uma peça configurável

`TituloCatalogo` sempre mostra o mesmo texto. Para exibir títulos diferentes sem copiar o componente, o pai envia dados de entrada. Esses dados formam as props, abreviação de properties, ou propriedades.

O PAI USA O MESMO COMPONENTE COM VALORES DIFERENTES

```
<TituloCatalogo texto="Produtos em destaque" />
<TituloCatalogo texto="Ofertas da semana" />
```

Em cada uso, React reúne os valores em um objeto. No primeiro caso, o componente recebe algo equivalente a `{ texto: "Produtos em destaque" }`. A comparação serve para entender o formato; continue usando `<TituloCatalogo />` no JSX em vez de chamar a função manualmente.

Etapas	O que acontece
1. Pai	App escreve a prop <code>texto</code> no uso de <code>TituloCatalogo</code> .

Etapa	O que acontece
2. React	Cria o objeto de props para aquela renderização.
3. Filho	TituloCatalogo recebe o objeto e lê texto.
4. Tela	O valor é inserido no h2 por meio de {texto}.

Por que o TypeScript pede um contrato?

ERRO PROPOSITAL - PROPS AINDA SEM TIPO

```
function TituloCatalogo(props) {
  return <h2>{props.texto}</h2>;
}
```

Com a configuração strict, TypeScript informa que props possui tipo implícito. Implícito significa não declarado. Sem o contrato, não sabemos se texto é obrigatório, se deve ser string ou se outra propriedade existe.

SRC/COMPONENTS/TITULOCATALOGO.TSX

```
type TituloCatalogoProps = {
  texto: string;
};

export function TituloCatalogo({ texto }: TituloCatalogoProps) {
  return <h2>{texto}</h2>;
}
```

TituloCatalogoProps descreve o objeto inteiro. texto: string exige uma propriedade chamada texto contendo texto. { texto } no parâmetro é desestruturação: retira a propriedade do objeto para usá-la diretamente.

ATENÇÃO Props são somente leitura. O filho usa os dados recebidos; ele não altera o objeto de props. Quando uma ação precisa mudar dados mantidos pelo pai, o filho chama uma função recebida por prop.

► VÍDEO DE APOIO [React 19 Tutorial - Props, Codevolution \(9 min 36 s\)](#)

Mostra o fluxo de valores entre componentes. Está em inglês; ative as legendas automáticas em português. Compare o exemplo com TituloCatalogoProps.

VAMOS PRATICAR Acrescente a prop quantidade: number a TituloCatalogoProps e mostre o texto Produtos em destaque (2). Atualize os dois usos no componente pai e observe o erro enquanto uma prop estiver ausente.

DUAS FORMAS VÁLIDAS, UMA ESCOLHA CONSCIENTE

5. type e interface: nomes para contratos de objeto

Um contrato de objeto declara quais propriedades existem e quais tipos são aceitos. type e interface podem nomear esse formato. Para props simples, as duas formas funcionam; o importante é escolher um padrão e manter consistência no projeto.

CONTRATO ESCRITO COM TYPE

```
type CardProdutoProps = {
  nome: string;
  preco: number;
};
```

O MESMO FORMATO ESCRITO COM INTERFACE

```
interface CardProdutoProps {
  nome: string;
  preco: number;
}
```

Necessidade	Escolha didática	Motivo
Props e objetos simples	type ou interface	Ambos descrevem o formato com clareza.
Alternativas com	type	Union é criada diretamente com type.

Necessidade	Escolha didática	Motivo
Combinar tipos com &	type	Intersection é expressa diretamente com &.
Extensão por extends	interface	A leitura de herança entre formatos fica explícita.

O sufixo Props não é palavra reservada. Ele é uma convenção de nome que informa ao leitor: este contrato descreve as entradas de um componente.

TROCAR QUALQUER TEXTO POR ALTERNATIVAS CONHECIDAS

6. Union types e tipos literais

Se status for apenas string, textos como disponível, disponível, DISPONIVEL ou um erro de digitação serão aceitos. Quando o sistema conhece as alternativas permitidas, podemos registrá-las no tipo.

TRÊS VALORES LITERAIS PERMITIDOS

```
type StatusProduto =
  | "disponível"
  | "esgotado"
  | "em_reposicao";
```

O símbolo | é lido como ou. StatusProduto aceita o texto exato disponível ou esgotado ou em_reposicao. Cada texto entre aspas funciona ao mesmo tempo como valor e como tipo literal.

UNION USADA EM PROPS

```
type SeloStatusProps = {
  status: StatusProduto;
};

export function SeloStatus({ status }: SeloStatusProps) {
  return <span>{status}</span>;
}
```

CONFIRA O RESULTADO <SeloStatus status="disponível" /> é aceito. <SeloStatus status="disponível" /> produz erro porque o acento cria outro texto.

VAMOS PRATICAR Crie type Tamanho = 'pequeno' | 'medio' | 'grande'. Use-o em uma prop tamanho e tente enviar 'gigante'. Leia a mensagem antes de corrigir.

MODELAR A AUSÊNCIA COM INTENÇÃO

7. Propriedades obrigatórias, opcionais e valores padrão

Uma propriedade obrigatória é necessária para o componente cumprir seu trabalho. Uma propriedade opcional pode faltar sem quebrar o resultado. O símbolo ? registra essa possibilidade no contrato.

VALOR PADRÃO PARA UMA PROP OPCIONAL

```
type CardProdutoProps = {
  nome: string;
  preco: number;
  descricao?: string;
};

export function CardProduto({
  nome,
  preco,
  descricao = "Descrição não informada",
}: CardProdutoProps) {
  return (
    <article>
      <h2>{nome}</h2>
      <p>R$ {preco.toFixed(2)}</p>
      <p>{descricao}</p>
    </article>
  );
}
```

descricao?: string significa string ou ausência. O valor padrão é aplicado quando descricao chega como undefined, isto é, sem valor definido. nome e preco continuam obrigatórios porque o card não consegue cumprir sua responsabilidade sem eles.

ATENÇÃO Não transforme uma prop em opcional apenas para apagar uma mensagem de erro. Primeiro pergunte se o componente realmente sabe funcionar quando esse dado falta.

DADOS DESCEM; EVENTOS SÃO COMUNICADOS

8. Callback: o filho comunica uma ação

Callback é uma função recebida para ser executada quando algo acontecer. O pai mantém a regra da aplicação e envia ao filho uma função. O filho chama essa função ao ocorrer um clique.

COMPONENTE FILHO

```
type BotaoAdicionarProps = {
  produtoId: number;
  aoAdicionar: (produtoId: number) => void;
};

export function BotaoAdicionar({
  produtoId,
  aoAdicionar,
}: BotaoAdicionarProps) {
  return (
    <button
      type="button"
      onClick={() => aoAdicionar(produtoId)}
    >
      Adicionar
    </button>
  );
}
```

(produtoId: number) => void é a assinatura da função: recebe um number e seu retorno não será usado. Em onClick, a arrow function cria uma função que será executada depois do clique. Escrever onClick={aoAdicionar(produtoId)} executaria a chamada durante a renderização.

COMPONENTE PAI DEFINE A REGRA

```
function App() {
  function adicionarAoCarrinho(id: number): void {
    console.log(`Adicionar produto ${id}`);
  }

  return (
    <BotaoAdicionar
      produtoId={1}
      aoAdicionar={adicionarAoCarrinho}
    />
  );
}
```

O pai envia produtoId e aoAdicionar. O filho não conhece o carrinho; ele apenas informa qual identificador foi clicado. Identificador, abreviado como id, é um valor usado para distinguir um item dos demais.

UM COMPONENTE PODE ENVOLVER OUTROS ELEMENTOS

9. children: composição de conteúdo

children representa o conteúdo colocado entre a abertura e o fechamento de um componente. Esse recurso permite criar uma estrutura visual sem obrigar o componente a conhecer antecipadamente cada parágrafo ou botão interno.

SRC/COMPONENTS/PAINEL.TSX

```
import type { ReactNode } from "react";

type PaineProps = {
  titulo: string;
  children: ReactNode;
};

export function Paine({ titulo, children }: PaineProps) {
  return (
    <section>
      <h2>{titulo}</h2>
      <div>{children}</div>
    </section>
  );
}
```

CONTEÚDO FORNECIDO PELO COMPONENTE PAI

```
<Paine titulo="Resumo do carrinho">
  <p>Você possui dois produtos.</p>
  <button type="button">Finalizar</button>
</Paine>
```

ReactNode é o tipo para conteúdos que React consegue renderizar, como texto, número, elemento, lista ou ausência. Paine controla a estrutura externa; o pai decide o conteúdo interno.

MAP TRANSFORMA DADOS EM INTERFACE

10. Listas: um componente para cada dado

SRC/COMPONENTS/LISTAPRODUTOS.TSX

```
type Produto = {
  id: number;
  nome: string;
};

type ListaProdutosProps = {
  produtos: Produto[];
};

export function ListaProdutos({ produtos }: ListaProdutosProps) {
  return (
    <ul>
      {produtos.map((produto) => (
        <li key={produto.id}>{produto.nome}</li>
      ))}
    </ul>
  );
}
```

Produto[] significa lista de objetos Produto. map percorre a lista; em cada passagem, produto representa o item atual; o retorno cria um elemento para esse item. O resultado do map é uma nova lista de elementos JSX.

key é uma identificação usada pelo React para relacionar cada dado ao elemento renderizado. Um id estável continua representando o mesmo produto mesmo quando a ordem muda. key é lida pelo React e não chega ao componente como prop comum.

VAMOS PRATICAR Crie uma lista de três categorias com id e nome. Renderize uma li para cada categoria e use categoria.id como key. Explique o papel de map, categoria e key.

O SÍMBOLO & É LIDO COMO E

11. Intersection: reunir contratos existentes

Intersection type combina exigências. Ela é útil quando dois contratos já possuem significados próprios e um novo tipo precisa cumprir os dois.

PRODUTORESUMIDO PRECISA DOS DOIS FORMATOS

```
type Identificado = {
  id: number;
};

type Nomeado = {
  nome: string;
};

type ProdutoResumido = Identificado & Nomeado;

const produto: ProdutoResumido = {
  id: 1,
  nome: "Teclado",
};
```

Identificado exige id. Nomeado exige nome. ProdutoResumido precisa de id e nome. O símbolo & não escolhe uma alternativa; ele soma as exigências.

PRIMEIRO A LÓGICA; DEPOIS O NOME TÉCNICO

12. Verificar a alternativa antes de usar

Uma union também pode reunir objetos com formatos diferentes. Antes de acessar uma propriedade exclusiva, o programa precisa descobrir qual formato está presente.

TRÊS FORMATOS POSSÍVEIS

```
type Resultado =
  | { estado: "carregando" }
  | { estado: "sucesso"; quantidade: number }
  | { estado: "erro"; mensagem: string };
```

estado	O que existe nesse objeto
carregando	Somente estado.
sucesso	estado e quantidade.
erro	estado e mensagem.

CADA CONDIÇÃO ELIMINA ALTERNATIVAS

```
function AvisoResultado({ resultado }: { resultado: Resultado }) {
  if (resultado.estado === "carregando") {
    return <p>Carregando...</p>;
  }

  if (resultado.estado === "erro") {
    return <p>Erro: {resultado.mensagem}</p>;
  }

  return <p>{resultado.quantidade} produtos.</p>;
}
```

Antes do primeiro if, resultado pode ter três formatos. Dentro do bloco carregando, sobra apenas o primeiro. Dentro do bloco erro, mensagem existe. No último return, as duas primeiras alternativas já foram encerradas, então resta sucesso e quantidade pode ser usada.

AGORA PODEMOS DAR O NOME Essa redução de possibilidades depois de uma verificação chama-se narrowing, ou estreitamento do tipo. estado é a etiqueta que permite distinguir cada alternativa; por isso, Resultado é uma união discriminada.

ATENÇÃO A verificação não existe apenas para satisfazer o TypeScript. Ela representa uma pergunta real da aplicação: o dado ainda carrega, chegou com sucesso ou terminou com erro?

REUTILIZAR SEM ABANDONAR A SEGURANÇA

13. Generic: deixar um espaço para o tipo real

Imagine um envelope com formato fixo: ele sempre informa sucesso e sempre carrega dados, mas o conteúdo pode ser uma lista de produtos, um usuário ou outra informação. Um generic cria um espaço que será preenchido pelo tipo real.

T É PREENCHIDO POR PRODUTO[]

```
type RespostaApi<T> = {
  dados: T;
  sucesso: boolean;
};

type Produto = {
  id: number;
  nome: string;
};

const resposta: RespostaApi<Produto[]> = {
  dados: [
    { id: 1, nome: "Teclado" },
    { id: 2, nome: "Mouse" },
  ],
  sucesso: true,
};
```

API significa Application Programming Interface, ou Interface de Programação de Aplicações. Em `RespostaApi<T>`, `T` é uma variável de tipo. Ao escrever `RespostaApi<Produto[]>`, cada ocorrência de `T` passa a significar `Produto[]`.

VOCÊ SABIA? Você já encontra generics em bibliotecas mesmo antes de criá-los: `useParams<{ produtoId: string }>()` informa qual formato esperamos nos parâmetros da rota.

► **VÍDEO DE APOIO** [Learn TypeScript Generics in 13 Minutes, Web Dev Simplified \(13 min\)](#)

Aprofundamento opcional e visual. Está em inglês; ative legendas automáticas em português e relacione o exemplo ao espaço `T` de `RespostaApi<T>`.

RELACIONAR URL E INTERFACE

14. Rotas: páginas também são componentes

URL significa Uniform Resource Locator, o endereço mostrado pelo navegador. Uma rota é uma regra que relaciona uma URL a um componente. Na TechStore, `/` pode mostrar Início e `/sobre` pode mostrar Sobre.

A aplicação continuará carregando o mesmo `index.html` e o React trocará o componente de página conforme a URL. Esse modelo é chamado Single Page Application (SPA), ou Aplicação de Página Única. Isso não significa uma única tela; significa que a navegação principal acontece dentro da aplicação já carregada.

Criar os componentes de página

SRC/PAGES/INICIO.TSX

```
export function Inicio() {
  return (
    <main>
      <h1>TechStore 2026</h1>
      <p>Conheça nossos produtos.</p>
    </main>
  );
}
```

SRC/PAGES/SOBRE.TSX

```
export function Sobre() {
  return (
    <main>
      <h1>Sobre a TechStore</h1>
      <p>Projeto didático criado com React e TypeScript.</p>
    </main>
  );
}
```

Início e Sobre são componentes comuns. Chamamos de página porque cada um será associado a uma rota e representará o conteúdo principal daquele endereço.

Ativar o roteador no ponto de entrada

SRC/MAIN.TSX

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router";
import App from "./App";
import "./index.css";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>,
);
```

BrowserRouter observa a URL do navegador e disponibiliza informações de navegação aos componentes colocados dentro dele. App precisa ficar dentro de BrowserRouter para usar Routes, Route e Link.

Relacionar caminhos e páginas

SRC/APP.TSX

```
import { Link, Route, Routes } from "react-router";
import { Inicio } from "./pages/Inicio";
import { Sobre } from "./pages/Sobre";

function App() {
  return (
    <>
      <header>
        <nav aria-label="Navegação principal">
          <Link to="/">Início</Link>
          <Link to="/sobre">Sobre</Link>
        </nav>
      </header>

      <Routes>
        <Route path="/" element={<Inicio />} />
        <Route path="/sobre" element={<Sobre />} />
        <Route path="*" element={<h1>Página não encontrada</h1>} />
      </Routes>
    </>
  );
}

export default App;
```

Recurso	Responsabilidade
Link	Altera a URL e solicita navegação sem recarregar todo o documento.
Routes	Examina os componentes Route e escolhe a correspondência atual.
Route	Relaciona um path, ou caminho, ao element que será renderizado.
path='*'	Representa qualquer endereço que não combinou com as rotas anteriores.

ATENÇÃO Use Link para navegação interna da aplicação. Uma tag a comum solicita outro carregamento do documento. Links externos, como documentação e vídeos, continuam usando a.

► **VÍDEO DE APOIO** [React Router - Configuring Routes, Codevolution \(7 min 52 s\)](#)

Explicação visual de Routes e Route. O vídeo usa a linha 6, mas os conceitos declarativos permanecem válidos. Para o código atual da linha 8, siga os imports desta apostila. Ative legendas em português.

VAMOS PRATICAR Crie `src/pages/Contato.tsx`, adicione `<Link to='/contato'>` e registre a rota `/contato`. Digite também `/nao-existe` para conferir a rota com `*`.

O ENDEREÇO TAMBÉM PODE CARREGAR UM DADO

15. Rota dinâmica, useParams e useNavigate

Uma rota dinâmica possui uma parte variável. Em `/produtos/2`, o número 2 identifica o produto. No caminho `/produtos/:produtoId`, os dois-pontos informam que `produtoId` é um parâmetro da URL.

ACRESCENTAR DENTRO DE ROUTES

```
<Route
  path="/produtos/:produtoId"
  element={<DetalheProduto />}
/>
```

Hook é uma função do ecossistema React cujo nome começa com `use` e fornece um recurso a um componente. `useParams` lê parâmetros da rota atual. `useNavigate` fornece uma função para navegar por código, por exemplo depois de um clique.

SRC/PAGES/DETALHEPRODUTO.TSX

```
import { useNavigate, useParams } from "react-router";

export function DetalheProduto() {
  const { produtoId } = useParams<{ produtoId: string }>();
  const navigate = useNavigate();

  if (!produtoId) {
    return <p>Identificador não informado.</p>;
  }

  const id = Number(produtoId);

  if (Number.isNaN(id)) {
    return <p>Identificador inválido.</p>;
  }

  return (
    <main>
      <h1>Produto {id}</h1>
      <button type="button" onClick={() => navigate("/")}>
        Voltar ao início
      </button>
    </main>
  );
}
```

Parâmetros de URL chegam como `string` ou podem estar ausentes. O `generic` informa o nome esperado, mas `produtoId` ainda precisa ser verificado. Depois de `if (!produtoId)`, TypeScript estreita o tipo para `string`. `Number` converte o texto em número; `Number.isNaN` verifica se a conversão falhou.

LIGAÇÃO ENTRE OS ASSUNTOS A rota fornece `string | undefined`; a condição verifica a alternativa; o `narrowing` mantém `string`; a conversão produz `number`; o componente usa o `id` com segurança.

MONTAR A ÁRVORE COMPLETA

16. Laboratório integrado: catálogo tipado com rotas

Agora os conceitos serão reunidos. Digite os arquivos na ordem apresentada. Não copie todos de uma vez: salve e confira o navegador depois de cada etapa.

SRC/TYPES/PRODUTO.TS

```
export type StatusProduto =
  | "disponivel"
  | "esgotado"
  | "em_reposicao";

export type Produto = {
  id: number;
  nome: string;
  preco: number;
  status: StatusProduto;
};
```

Este arquivo contém apenas contratos, por isso usa .ts. export permite que componentes e páginas usem Produto e StatusProduto.

SRC/DATA/PRODUTOS.TS

```
import type { Produto } from "../types/produto";

export const produtos: Produto[] = [
  { id: 1, nome: "Teclado", preco: 149.9, status: "disponivel" },
  { id: 2, nome: "Mouse", preco: 89.9, status: "esgotado" },
  { id: 3, nome: "Monitor", preco: 899, status: "em_reposicao" },
];
```

Produto[] obriga cada objeto da lista a cumprir o contrato. data significa dados; separar a lista evita repetir os mesmos produtos nas páginas.

SRC/COMPONENTS/CARDPRODUTO.TSX

```
import { Link } from "react-router";
import type { Produto } from "../types/produto";

type CardProdutoProps = {
  produto: Produto;
  aoAdicionar: (produtoId: number) => void;
};

export function CardProduto({ produto, aoAdicionar }: CardProdutoProps) {
  const disponivel = produto.status === "disponivel";

  return (
    <article>
      <h2>{produto.nome}</h2>
      <p>R$ {produto.preco.toFixed(2)}</p>
      <p>Status: {produto.status}</p>
      <Link to={` /produtos/${produto.id}`}>Ver detalhes</Link>
      <button
        type="button"
        disabled={!disponivel}
        onClick={() => aoAdicionar(produto.id)}
      >
        Adicionar
      </button>
    </article>
  );
}
```

CardProduto recebe um objeto completo e um callback. O Link monta uma URL diferente para cada id. disabled impede o clique quando o status não é disponivel.

SRC/COMPONENTS/LISTAPRODUTOS.TSX

```
import { CardProduto } from "../CardProduto";
import type { Produto } from "../types/produto";

type ListaProdutosProps = {
  produtos: Produto[];
  aoAdicionar: (produtoId: number) => void;
};

export function ListaProdutos({
  produtos,
  aoAdicionar,
}: ListaProdutosProps) {
  if (produtos.length === 0) {
    return <p>Nenhum produto cadastrado.</p>;
  }

  return (
    <section>
      {produtos.map((produto) => (
        <CardProduto
          key={produto.id}
          produto={produto}
          aoAdicionar={aoAdicionar}
        />
      ))}
    </section>
  );
}
```

ListaProdutos organiza a coleção; CardProduto apresenta um item. Essa separação permite ler cada responsabilidade sem examinar toda a página.

SRC/PAGES/INICIO.TSX

```
import { ListaProdutos } from "../components/ListaProdutos";
import { produtos } from "../data/produtos";

export function Inicio() {
  function adicionarAoCarrinho(produtoId: number): void {
    console.log(`Adicionar produto ${produtoId}`);
  }

  return (
    <main>
      <h1>TechStore 2026</h1>
      <ListaProdutos
        produtos={produtos}
        aoAdicionar={adicionarAoCarrinho}
      />
    </main>
  );
}
```

Inicio é a página pai. Ela conhece os dados e a ação. ListaProdutos recebe as duas props, percorre os dados e repassa cada produto ao card.

SRC/PAGES/DETALHEPRODUTO.TSX

```
import { useNavigate, useParams } from "react-router";
import { produtos } from "../data/produtos";

export function DetalheProduto() {
  const { produtoId } = useParams<{ produtoId: string }>();
  const navigate = useNavigate();
  const id = Number(produtoId);
  const produto = produtos.find((item) => item.id === id);

  if (!produto) {
    return (
      <main>
        <h1>Produto não encontrado</h1>
        <button type="button" onClick={() => navigate("/")}>
          Voltar ao início
        </button>
      </main>
    );
  }

  return (
    <main>
      <h1>{produto.nome}</h1>
      <p>R$ {produto.preco.toFixed(2)}</p>
      <p>Status: {produto.status}</p>
      <button type="button" onClick={() => navigate("/")}>
        Voltar ao início
      </button>
    </main>
  );
}
```

find procura o primeiro item cujo id combina com a URL. O resultado é Produto | undefined. Depois de if (!produto), a alternativa undefined termina e o último bloco pode usar nome, preco e status com segurança.

SRC/PAGES/SOBRE.TSX E SRC/PAGES/NAOENCONTRADA.TSX

```
export function Sobre() {
  return (
    <main>
      <h1>Sobre</h1>
      <p>Catálogo didático com componentes e tipos.</p>
    </main>
  );
}

export function NaoEncontrada() {
  return (
    <main>
      <h1>Página não encontrada</h1>
      <p>Confira o endereço digitado.</p>
    </main>
  );
}
```

SRC/APP.TSX

```
import { Link, Route, Routes } from "react-router";
import { DetalheProduto } from "../pages/DetalheProduto";
import { Inicio } from "../pages/Inicio";
import { NaoEncontrada } from "../pages/NaoEncontrada";
import { Sobre } from "../pages/Sobre";

export default function App() {
  return (
    <>
      <header>
        <nav aria-label="Navegação principal">
          <Link to="/">Inicio</Link>
          <Link to="/sobre">Sobre</Link>
        </nav>
      </header>

      <Routes>
        <Route path="/" element={<Inicio />} />
        <Route path="/sobre" element={<Sobre />} />
        <Route path="/produtos/:produtoId" element={<DetalheProduto />} />
        <Route path="*" element={<NaoEncontrada />} />
      </Routes>
    </>
  );
}
```

CONFIRA O RESULTADO Início mostra três cards; Sobre altera a URL e o conteúdo; Ver detalhes abre o produto correto; um id inexistente mostra Produto não encontrado; outra URL mostra Página não encontrada.

TERMINAL - VALIDAÇÃO FINAL

```
npm run build
npm run lint
```

build verifica o TypeScript e prepara a versão de produção. lint analisa padrões e possíveis problemas no código. Corrija qualquer mensagem antes de considerar o laboratório concluído.

PROCURE A CAUSA ANTES DE REPETIR COMANDOS

17. Diagnóstico de mensagens comuns

Mensagem ou sintoma	Causa provável	Primeira verificação
package.json não encontrado	Terminal fora da pasta do projeto.	Execute cd techstore-types e confira o arquivo.
vite não reconhecido	Dependências ainda não instaladas.	Execute npm install dentro do projeto.
Apareceu create-vite 9.x	É a versão do gerador, não do Vite interno.	Use npx vite --version para conferir o projeto.
Cannot find module react-router	Pacote de rotas ausente.	Execute npm install react-router.
useRoutes may be used only in a Router	App está fora de BrowserRouter.	Revise src/main.tsx.
Property 'x' is missing	Prop obrigatória não foi enviada.	Compare uso do componente e contrato Props.
Possibly undefined	Uma ausência ainda não foi verificada.	Use condição, valor padrão ou torne o dado obrigatório.
No routes matched location	Nenhuma rota corresponde ao endereço.	Confira path e mantenha uma rota com *.

O MODELO MENTAL COMPLETO

Resumo da aula

- create-vite 9.1 gera a base; Vite 8.2 permanece como ferramenta do projeto.
- main.tsx inicia o React e fornece o BrowserRouter; App organiza navegação e rotas.
- Componentes separam responsabilidades e são combinados por composição.
- Props formam o objeto de entrada enviado do pai para o filho.
- type e interface nomeiam contratos; unions e literais limitam alternativas.
- Callbacks comunicam ações; children permite fornecer conteúdo interno.
- map transforma dados em elementos e key preserva a identidade dos itens.
- Condições reduzem alternativas; essa redução recebe o nome narrowing.
- Rotas relacionam URLs a componentes; useParams lê parâmetros e useNavigate navega por código.

VERIFIQUE O QUE APRENDEU

Exercícios

Responda antes de consultar o gabarito. Nas questões práticas, registre o comando usado e a saída observada.

1. Explique a diferença entre create-vite e Vite e informe a função de cada um.

Resposta: _____

2. Escreva o comando completo para criar techstore-types com o template react-ts.

Resposta: _____

3. Explique o caminho index.html → main.tsx → App.tsx.

Resposta: _____

4. Por que separar CardProduto e ListaProdutos melhora a leitura da aplicação?

Resposta: _____

5. Crie um componente Rodape sem props, exporte-o e use-o em App.

Resposta: _____

6. Qual diferença o React reconhece entre <main> e <Inicio />?

Resposta: _____

7. Crie TituloProps com texto string e quantidade number e use o contrato em um componente.

Resposta: _____

8. Por que props seguem do pai para o filho, mas uma ação do filho pode chegar ao pai por callback?

Resposta: _____

9. Crie type Tema = 'claro' | 'escuro' e use-o em uma prop.

Resposta: _____

10. Quando uma prop deve ser opcional? Por que torná-la opcional só para apagar erro é perigoso?

Resposta: _____

11. Explique a função de ReactNode no contrato de children.

Resposta: _____

12. Por que produto.id é uma key melhor que a posição do item na lista?

Resposta: _____

13. Qual diferença existe entre A | B e A & B?

Resposta: _____

14. Por que resultado.mensagem só pode ser usado depois de confirmar estado === 'erro'?

Resposta: _____

15. O que T representa em RespostaApi<Produto[]>?

Resposta: _____

16. Crie a rota /contato, seu componente de página e um Link para ela.

Resposta: _____

17. Explique BrowserRouter, Routes, Route e Link sem usar apenas os nomes.

Resposta: _____

18. Por que useParams devolve texto mesmo quando o parâmetro parece um número?

Resposta: _____

19. Crie a rota /categorias/:categoriaId e mostre o parâmetro depois de verificar sua existência.

Resposta: _____

20. Adicione categoria aos produtos, mostre-a no card e crie uma página de detalhe que trate id inexistente.

Resposta: _____

APRENDA TAMBÉM COM A CORREÇÃO

Gabarito comentado

1. create-vite é o gerador executado durante a criação e está na linha 9.1. Vite é a ferramenta instalada no projeto para dev e build e está na linha 8.2.
2. npm create vite@latest techstore-types -- --template react-ts.
3. index.html oferece o elemento root; main.tsx localiza root, inicia React e renderiza App; App organiza a interface, a navegação e as rotas.
4. ListaProdutos cuida da coleção e do map; CardProduto apresenta um item. Cada componente fica com uma responsabilidade identificável e pode mudar com menos impacto.
5. Crie export function Rodape() { return <footer>TechStore 2026</footer>; }, importe em App e use <Rodape />.
6. main começa em minúscula e representa um elemento HTML. Inicio começa em maiúscula e faz React procurar a função componente Inicio.
7. type TituloProps = { texto: string; quantidade: number }; e function Titulo({ texto, quantidade }: TituloProps) { return <h2>{texto} ({quantidade})</h2>; }.
8. O pai envia uma função por prop. O filho não altera os dados do pai; ele chama o callback e informa que a ação aconteceu. O pai decide como reagir.
9. type Tema = 'claro' | 'escuro'; depois use type Props = { tema: Tema }.
10. Ela deve ser opcional quando o componente produz um resultado válido sem o dado. Tornar tudo opcional transfere falhas para condições internas e enfraquece o contrato.
11. ReactNode descreve os tipos de conteúdo que React pode renderizar dentro do componente, como elementos, textos, listas ou ausência.
12. O id representa o mesmo produto mesmo se a ordem mudar. A posição pode passar a representar outro item depois de inserções, exclusões ou ordenação.
13. A | B aceita uma alternativa ou outra. A & B exige ao mesmo tempo as propriedades de A e de B.
14. Antes da condição, resultado também pode ser carregando ou sucesso, formatos sem mensagem. A verificação estreita o tipo para a alternativa erro.
15. T é substituído por Produto[]. Portanto, a propriedade dados deve receber uma lista de produtos.
16. Crie Contato.tsx, importe-o em App, adicione <Link to='/contato'>Contato</Link> e <Route path='/contato' element={<Contato />} />.
17. BrowserRouter fornece o contexto da URL; Routes escolhe a correspondência; Route relaciona path e elemento; Link solicita navegação interna.
18. A URL é texto no navegador. Por isso, productId chega como string ou ausência e precisa ser convertido quando a regra exige number.
19. Registre path='/categorias/:categoriaId', use useParams, confirme if (!categoriaId) e só depois renderize o valor.
20. Acrescente categoria: string ao contrato e aos dados, renderize produto.categoria no card e use find mais uma condição de ausência na página de detalhe.

ANTES DE ENCERRAR

Lista de verificação final

- Consigo explicar boilerplate, template, create-vite e Vite sem confundir suas versões;
- Consigo criar, instalar, executar e validar o projeto pelo terminal;
- Consigo explicar o fluxo entre index.html, main.tsx e App.tsx;
- Consigo identificar responsabilidades antes de separar componentes;
- Consigo criar, exportar, importar e compor componentes funcionais;
- Consigo distinguir elemento HTML de componente React pela escrita;
- Consigo explicar componente pai, componente filho e árvore da interface;
- Consigo modelar props com type ou interface;
- Consigo usar unions, literais, opcionais e valores padrão com intenção;
- Consigo tipar callbacks e compreender o momento da chamada em onClick;
- Consigo usar children com ReactNode;
- Consigo renderizar listas com map e key estável;
- Consigo distinguir union de intersection;
- Consigo verificar alternativas antes de acessar propriedades específicas;
- Consigo reconhecer um generic e preencher T com um tipo real;
- Consigo criar rotas estáticas, dinâmica e de página não encontrada;
- Consigo usar Link, useParams e useNavigate com explicação do papel de cada um;
- Consigo concluir npm run build e npm run lint sem erros.

BIBLIOGRAFIA E ATUALIZAÇÃO

Referências oficiais

- [Vite - primeiros passos](#)
- [Vite - política de versões](#)
- [npm - pacote create-vite](#)
- [npm - pacote Vite](#)
- [React - seu primeiro componente](#)
- [React - importar e exportar componentes](#)
- [React - passando props](#)
- [React - composição com children](#)
- [React - renderizando listas](#)
- [React Router - modo declarativo](#)
- [React Router - navegação](#)
- [React Router - parâmetros de URL](#)
- [TypeScript - tipos de objetos](#)
- [TypeScript - narrowing](#)
- [TypeScript - generics](#)

Documentação consultada e procedimentos validados em 11 de agosto de 2026.

AUTORIA E DIREITOS | Alexandre Carlos de Jesus • GitHub: [alecarlosjesus](#) • LinkedIn: [alexandre-carlos-de-jesus](#) • © 2026
Alexandre Carlos de Jesus. Todos os direitos reservados. Reprodução, distribuição, adaptação ou comercialização, total ou parcial, somente com autorização prévia por escrito, salvo permissões legais. Marcas e logotipos pertencem aos respectivos titulares.